There may be a cutoff value, after which it isn't so important to express how large a datapoint is. For instance, let's say these datapoints represent the number of responses for a survey, and it's deemed a success if there are more than 500 responses. We may only want to show the range of the data values between 0 and 500, while emphasizing the variation at the 0 to 100 level with a scale like this:

```
var yScale = d3.scale.linear()
.domain([0,100,500]).range([0,50,100]);
```

You may think that's enough to draw a new chart that caps the bars at a maximum height of 100 if the datapoint has a value over 500. This isn't the default behavior for scales in D3, though. In figure 2.19 you can see what would happen running the draw code with that scale.

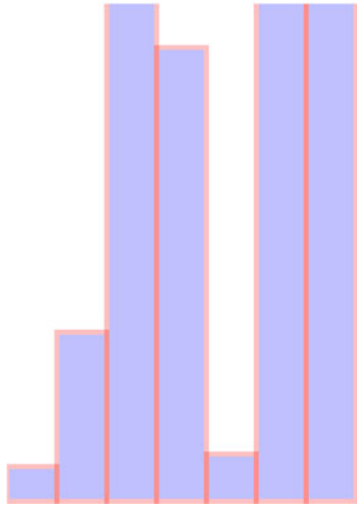


Figure 2.19 A bar chart drawn with a linear scale where the maximum value in the domain is lower than the maximum value in the dataset

Notice the rectangles are still drawn above the canvas, as evidenced by the lack of a border on the top of the four rectangles with values over 500. We can confirm this is happening by putting a value greater than 500 into the scale function we've created:

```
yScale(1000);    <-- Returns 162.5
```

By default, a D3 scale continues to extrapolate values greater than the maximum domain value and less than the minimum domain value. If we want it to set all such values to the maximum (for greater) or minimum (for lesser) range value, then we need to use the `.clamp()` function:

```
var yScale = d3.scale.linear()
               .domain([0,100,500])
               .range([0,50,100])
               .clamp(true);
```

Running the draw code now produces rectangles that have a maximum value of 100 for height and position, as shown in figure 2.20.

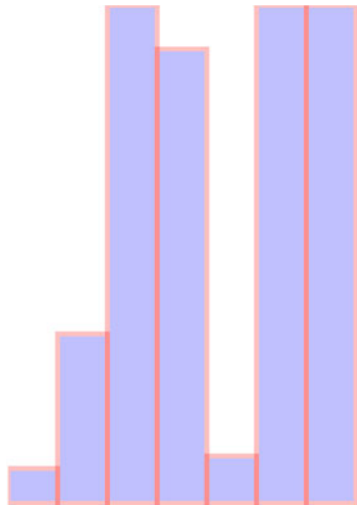


Figure 2.20 A bar chart drawn with values in the dataset greater than the maximum value of the domain of the scale, but with the `clamp()` function set to `true`

We can confirm this by plugging a value into `yScale()` that's greater than 500:

```
yScale(1000);    <-- Returns 100
```

Scale functions are key to determining position, size, and color of elements in data visualization. As